

What is The Output:

1-

What does this print?

```
type Point struct { X, Y int } func main() { p1 :=  
Point{1, 2}  
p2 := p1  
p2.X = 100  
fmt.Println(p1.X, p2.X) }
```

1 100

الـ Structs في لغة Go هي Value Types وليست Reference Types

2-

```
type Counter struct { count int }
func (c Counter) IncByValue() { c.count++ }
func (c *Counter) IncByPointer() { c.count++ }
func main() {
    c := Counter{}
    c.IncByValue()
    c.IncByPointer()
    fmt.Println(c.count)
}
```

1

تم إنشاء المتغير **c** بالقيم الافتراضية **Counter{}**، فتصبح قيمة **c.count** تبدأ من 0
الدالة **IncByPointer()** تستخدم **Pointer Receiver (c *Counter)**

عملية **c.count++** تعدل القيمة في العنوان الأصلي مباشرة، فتزيد **c.count** من 0 إلى 1.

3-

```
type Point struct { X, Y int }  
func main() {  
    p1 := Point{1, 2}  
    p2 := Point{1, 2}  
    fmt.Println(p1 == p2)  
}
```

true

تساوی القيم والنوع والحجم

4-

```
type Box struct { Items []int }
func main() {
    b1 := Box{Items: []int{1, 2, 3}}
    b2 := b1
    b2.Items[0] = 99
    fmt.Println(b1.Items[0])
}
```

99

• بما أن **b1.Items** تشير لنفس العنوان **b2**، فإن القيمة عند طباعة **b1.Items[0]** أصبحت **99**

5-

```
type Item struct { Val int }
func main() {
    items := []Item{{1}, {2}, {3}}
    for _, it := range items { it.Val *= 10 }
    fmt.Println(items[0].Val, items[1].Val, items[2].Val)
}
```

1 2 3

- `it.Val *= 10` يعدل في المتغير المؤقت المحلي `it` فقط، ولا يعدل في العناصر الأصلية داخل الـ `Slice` المسمى `items`
- لتعديل العناصر الأصلية داخل الـ `Slice` ، يجب التعديل باستخدام `Index` مباشرة مثل `items[i].Val *= 10`

6-

```
type Engine struct {  
    HP int  
}  
type Car struct {  
    *Engine  
}  
func main() {  
    c := Car{}  
    fmt.Println(c.Engine == nil)  
}
```

true

بما أن المؤشر **c.Engine** لم يتم تهيئته بعناوين ذاكرة (**nil**) ، فإن الشرط **c.Engine == nil** يتحقق ويكون الناتج **true**

7-

```
type Config struct {  
    Settings map[string]string  
}  
  
func main() {  
    var c Config  
    c.Settings["key"] = "value"  
    fmt.Println(c.Settings["key"])  
}
```

panic: assignment to entry in nil map

- القيمة الافتراضية للـ **map** في لغة **Go** هي **nil**
- **(Assignment)** إلى **nil map** غير مسموحة مطلقاً في **Go** وتتسبب فوراً في

panic: assignment to entry in nil map

```

func safeDivide(a, b int) (result int) {
    defer func() {
        if r := recover(); r != nil {
            result = -1
        }
    }()
    result = a / b
    return
}

func main() {
    fmt.Println(safeDivide(10, 2))
    fmt.Println(safeDivide(10, 0))
}

```

5
-1

الاستدعاء الثاني: safeDivide(10, 0)

- عند محاولة تنفيذ $10 / 0$ تحدث مشكلة قسمة على صفر وتطلق لغة Go استثناء حرجاً وهو **runtime panic: integer divide by zero**.
- يتوقف تنفيذ الدالة فوراً وينتقل التحكم إلى دالة الـ **defer** المسجلة سابقاً
- داخل الـ **defer** تنفذ الدالة **recover()** التي تقوم بالنقاط الـ **panic** وإيقاف انهيار البرنامج
- تحقق الشرط **r != nil** ويتم تغيير قيمة متغير إرجاع الدالة **result** إلى **-1**
- ترجع الدالة القيمة **-1** بنجاح

Write Go Code To Solve These Problems

1. Employee Directory

Create an Employee struct with ID, name,

department, salary, and hire date. Requirements:

- Add methods to calculate years employed and apply a percentage raise.
- Reject negative salaries.
- Sort employees by salary and department.

in **EmployeeDirectory.go**

=====

2. Library Lending System

Model Book, Member, and Loan

structs. Requirements

- Track availability and due dates.
- Implement borrow and return methods.
- Prevent unavailable books from being borrowed.
- Return custom errors for invalid operations.

in **LibraryLendingSystem.go**

3. Embedded Address Book

Create Address, Contact, and BusinessContact

structs using embedding. Requirements:

- Promote address fields through embedding.
- Implement a String() string method.
- Search contacts by city or partial name.

in **EmbeddedAddressBook.go**

=====

4. JSON Product Catalog

Create a Product struct with JSON tags for SKU, name, price,

stock, and optional description. Requirements:

- Decode products from JSON.
- Validate price and stock values.
- Omit empty descriptions when encoding.
- Calculate total inventory value.

5.in **JSONProductCatalog.go**

6. Array Index Boundary Safeguard

Objective: Catch standard Go runtime panics (e.g., out-of-bounds array access).

Task: Write a function `getElement(slice []string, index int) (result string, err error)`.

1. Attempt to access `slice[index]` directly without checking `len(slice)` beforehand.
2. Use `defer` and `recover()` to capture any index out-of-range panic generated by the Go runtime.
3. If a panic occurs, set the named return value `err` to `fmt.Errorf("index %d out of bounds", index)`.
4. If no panic occurs, return the string element and `nil` error.

7. in **ArrayIndexBoundarySafeguard.go**

